

**LAPORAN PRAKTIKUM
PEMROGRAMAN II
MODUL 6**



Solid Principle dan Design Pattern

Oleh:

Muhammad Naufal Khalish

NIM. 2410817110004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
NOVEMBER 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN II
MODUL 6

Laporan Praktikum Pemrograman II Modul 6: Solid Principle dan Design Pattern ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Naufal Khalish
NIM : 2410817110004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Jovan Gilbert Natamasindah
NIM. 2310817310002

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
SOAL PRAKTIKUM	6
A. Source Code	7
B. Output Program.....	13
C. Pembahasan.....	21
TAUTAN GIT.....	23

DAFTAR GAMBAR

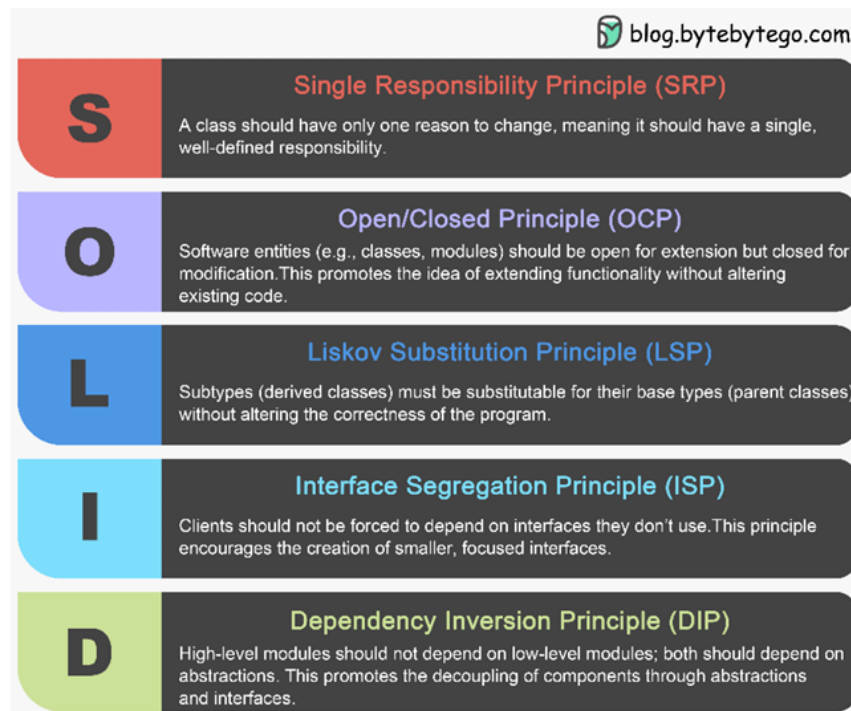
Gambar 1. Program File SOLID saat memilih pilihan 1	13
Gambar 2. Program File SOLID setelah memilih pilihan 1.....	13
Gambar 3. Program File SOLID saat memilih pilihan 2	14
Gambar 4. Program File SOLID memilih senjata di pilihan 2	14
Gambar 5. Program File SOLID setelah memilih pilihan 2.....	14
Gambar 6. Program File SOLID saat memilih pilihan 3	15
Gambar 7. Program File SOLID setelah memilih pilihan 3.....	15
Gambar 8. Program File SOLID saat memilih pilihan 4	15
Gambar 9. Program File Design Pattern saat memilih pilihan 1.....	16
Gambar 10. Program File Design Pattern memasukkan nama pada pilihan 1	16
Gambar 11. Program File Design Pattern setelah memilih pilihan 1.....	16
Gambar 12. Program File Design Pattern saat memilih pilihan 2.....	17
Gambar 13. Program File Design Pattern memilih senjata di pilihan 2.....	17
Gambar 14. Program File Design Pattern setelah memilih pilihan 2.....	17
Gambar 15. Program File Design Pattern saat memilih pilihan 3.....	18
Gambar 16. Program File Design Pattern memilih senjata di pilihan 3.....	18
Gambar 17. Program File Design Pattern memasukkan angka di pilihan 3	18
Gambar 18. Program File Design Pattern setelah memilih pilihan 3.....	19
Gambar 19. Program File Design Pattern saat memilih pilihan 4.....	19
Gambar 20. Program File Design Pattern menulis teks di pilihan 4.....	19
Gambar 21. Program File Design Pattern setelah memilih pilihan 4.....	20
Gambar 22. Program File Design Pattern saat memilih pilihan 5.....	20

DAFTAR TABEL

Table 1. Source Code UI.java Implementasi Single Responsibility Principle.....	7
Table 2. Source Code gabunganstaff.java Implementasi Open Closed Principle dan Liskov Substitution Principle.....	7
Table 3 Source Code Skill.java Implementasi Interface Segregation Principle	9
Table 4 Source Code gabunganweapon.java Implementasi Dependency Inversion Principle	9
Table 5. Source Code gabunganstaff.java Implementasi Creational Pattern - Factory Method	10
Table 6. Source Code gabungandecorator.java Implementasi Structural Pattern - Decorator.....	11
Table 7. Source Code gabunganskill.java Implementasi Behavioral Pattern - Strategy	12

SOAL PRAKTIKUM

- Berdasarkan tugas CLI template yang telah anda buat masing-masing sebelumnya. Sekarang ubah atau **lakukan refactor terhadap code CLI tersebut untuk menerapkan masing-masing prinsip SOLID**



- Kemudian setelah melakukan implementasi dari SOLID, selanjutnya silahkan **pilih masing-masing 1 design pattern dari 3 jenis design pattern untuk diimplementasikan ke code** tadi dari link ini <https://refactoring.guru/design-patterns/catalog> (kecuali singleton)

Catatan:

- Untuk format laprak sama seperti yang sebelum-sebelumnya
- Untuk bagian source code, pisah jadi dua bagian, yaitu berisikan implementasi SOLID dan berisikan implementasi SOLID dengan design pattern yang dipilih.
- Untuk bagian penjelasan, hanya menjelaskan di bagian mana masing-masing bagian SOLID diterapkan dan penjelasan design pattern yang dipilih dengan detail.

A. Source Code

Penerapan SOLID :

Table 1. Source Code UI.java Implementasi Single Responsibility Principle

```

1 package Praktikum.praktikum6.SOLID;
2
3 public class UI {
4     public void showPage(String content) {
5         System.out.println("=====
6         =====");
7         System.out.println(" ALBION ONLINE - STAFF INFO");
8         System.out.println("=====
9         =====\n");
10        System.out.println(content);
11        System.out.println("\n=====
12        =====");
13        System.out.println("
14                                >__<
15        ");
16        System.out.println("=====
17        =====\n");
18    }
19 }

```

Table 2. Source Code gabunganstaff.java Implementasi Open Closed Principle dan Liskov Substitution Principle

```
1 package Praktikum.praktikum6.SOLID;
2
3 public abstract class staff {
4     protected String name;
5     protected String description;
6     protected double baseDamage;
7     protected Skill skill;
8
9     public staff(String name, String description, double
baseDamage, Skill skill){
10         this.name=name;
11         this.description=description;
12         this.baseDamage=baseDamage;
13         this.skill= skill;
14     }
15
16     public void setSkill(Skill skill){
17         this.skill=skill;
```

18	}
19	
20	public String getName() {
21	return name;
22	}
23	
24	public String getDescription() {
25	return description;
26	}
27	
28	public double getBaseDamage() {
29	return baseDamage;
30	}
31	
32	public String getSkillInfo() {
33	return skill.getSkillName() + " --> " +
	skill.useSkill();
34	}
35	}

1	package Praktikum.praktikum6.SOLID;
2	
3	public class novicesFireStaff extends staff{
4	public novicesFireStaff() {
5	super("Novice Fire Staff", "Staff api dasar
	untuk pemula", 10.0, new noviceFireSkill());
6	
7	}
8	}

1	package Praktikum.praktikum6.SOLID;
2	
3	public class adeptHolyStaff extends staff{
4	public adeptHolyStaff(){
5	super("Adept Holy Staff", "Staff suci dengan
	kemampuan penyembuhan", 15, new adeptHolySkill());
6	}
7	}

1	package Praktikum.praktikum6.SOLID;
2	
3	public class expertGreatStaff extends staff{
4	public expertGreatStaff(){
5	super("Expert Great Staff", "Staff besar dengan
	kekuatan energi", 20, new expertGreatSkill());
6	}

7	} }
---	--------

Table 3 Source Code Skill.java Implementasi Interface Segregation Principle

1	package Praktikum.praktikum6.SOLID;
2	
3	public interface Skill {
4	String getSkillName();
5	String useSkill();
6	}

Table 4 Source Code gabunganweapon.java Implementasi Dependency Inversion Principle

1	package Praktikum.praktikum6.SOLID;
2	import java.util.List;
3	
4	public interface weaponRepository {
5	List<staff>findAll();
6	
7	}

1	package Praktikum.praktikum6.SOLID;
2	import java.util.List;
3	
4	public class weaponService {
5	private final weaponRepository repository;
6	
7	public weaponService(weaponRepository repository){
8	this.repository=repository;
9	}
10	
11	public List<staff> getAllWeapons(){
12	return repository.findAll();
13	}
14	}

Penerapan Design Pattern :

Table 5. Source Code gabunganstaff.java Implementasi Creational Pattern - Factory Method

```
1 package Praktikum.praktikum6.DesignPattern;
2
3 public class staffFactory {
4     public static staff createStaff(String type){
5         switch(type.toLowerCase()){
6             case "novice_fire":
7                 return new noviceFireStaff();
8             case "adept_holy":
9                 return new adeptHolyStaff();
10            case "expert_great":
11                return new expertGreatStaff();
12            default:
13                throw new
14                IllegalArgumentException("Unknowrn staff type");
15        }
16    }
17 }
```

```
1 package Praktikum.praktikum6.DesignPattern;
2
3 public class noviceFireStaff extends staff{
4     public noviceFireStaff(){
5         super("Novice Fire Staff",
6             "Staff api dasar untuk pemula",
7             10,
8             new noviceFireSkill());
9     }
10 }
```

```
1 package Praktikum.praktikum6.DesignPattern;
2
3 public class adeptHolyStaff extends staff{
4     public adeptHolyStaff(){
5         super("Adept Holy Staff",
6             "Staff dengan kemampuan penyembuhan",
7             14,
8             new adeptHolySkill());
9     }
10 }
```

1	package Praktikum.praktikum6.DesignPattern;
2	
3	public class expertGreatStaff extends staff{
4	public expertGreatStaff(){
5	super("Expert Great Staff",
6	"Staff besar dengan kekuatan energi
7	murni",
8	20,
9	new expertGreatSkill());
10	}

Table 6. Source Code gabungandecorator.java Implementasi Structural Pattern - Decorator

1	package Praktikum.praktikum6.DesignPattern;
2	
3	public abstract class skillDecorator implements Skill{
4	protected Skill wrappedSkill;
5	
6	public skillDecorator(Skill wrappedSkill) {
7	this.wrappedSkill = wrappedSkill;
8	}
9	}

1	package Praktikum.praktikum6.DesignPattern;
2	
3	public class buffDamageDecorator extends skillDecorator
4	{
5	private final int added;
6	public buffDamageDecorator(Skill skill, int added){
7	super(skill);
8	this.added=added;
9	}
10	
11	@Override
12	public String getSkillName() {
13	return wrappedSkill.getSkillName() + " + Buff(+"
14	+ added + ")";
15	}
16	@Override
17	public String useSkill() {
18	return wrappedSkill.useSkill() + " (Damage +" +
19	added + ")";
20	}

Table 7. Source Code gabunganskill.java Implementasi Behavioral Pattern - Strategy

```

1 package Praktikum.praktikum6.DesignPattern;
2
3 public interface Skill {
4     String getSkillName();
5     String useSkill();
6 }

```

```

1 package Praktikum.praktikum6.DesignPattern;
2
3 public class noviceFireSkill implements Skill {
4     @Override
5     public String getSkillName() {
6         return "Fire Bolt";
7     }
8
9     @Override
10    public String useSkill() {
11        return "Menembakkan api kecil ke musuh";
12    }
13 }

```

```

1 package Praktikum.praktikum6.DesignPattern;
2
3 public class adeptHolySkill implements Skill{
4     @Override
5     public String getSkillName() {
6         return "Holy Bless";
7     }
8
9     @Override
10    public String useSkill() {
11        return "Memulihkan HP sekutu";
12    }
13
14 }

```

```

1 package Praktikum.praktikum6.DesignPattern;
2
3 public class expertGreatSkill implements Skill{
4     @Override
5     public String getSkillName() {
6         return "Soul Beam";
7     }
8
9     @Override

```

10	public String useSkill() {
11	return "Menembakkan sinar energi besar";
12	}
13	}

B. Output Program

```
=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Username
2. Lihat Deskripsi Senjata
3. Pesan Feedback
4. Keluar

=====
>_<
=====

>> 1
>> Masukkan Username: Naufal
```

Gambar 1. Program File SOLID saat memilih pilihan 1

```
=====
ALBION ONLINE - STAFF INFO
=====

halo Naufal! (ENTER untuk kembali)

=====
>_<
=====
```

Gambar 2. Program File SOLID setelah memilih pilihan 1

```

=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Useername
2. Lihat Deskripsi Senjata
3. Pesan Feedback
4. Keluar

=====
>_<
=====

>> 2

```

Gambar 3. Program File SOLID saat memilih pilihan 2

```

=====
ALBION ONLINE - STAFF INFO
=====

Pilih Senjata:

1. Novice Fire Staff
2. Adept Holy Staff
3. Expert Great Staff

=====
>_<
=====

>> 1

```

Gambar 4. Program File SOLID memilih senjata di pilihan 2

```

=====
ALBION ONLINE - STAFF INFO
=====

Nama : Novice Fire Staff
Deskripsi : Staff api dasar untuk pemula
baseDamage : 10.0
Skill : Fire Bolt --> Menembakkan api kecil ke musuh

(ENTER untuk kembali

=====
>_<
=====

```

Gambar 5. Program File SOLID setelah memilih pilihan 2

```
=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Useername
2. Lihat Deskripsi Senjata
3. Pesan Feedback
4. Keluar

=====
>__<
=====

>> 3
>> Tulis feedback: Terima Kasih
```

Gambar 6. Program File SOLID saat memilih pilihan 3

```
=====
ALBION ONLINE - STAFF INFO
=====

Feedback diterima:

Terima Kasih

(ENTER untuk kembali)

=====
>__<
=====
```

Gambar 7. Program File SOLID setelah memilih pilihan 3

```
=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Useername
2. Lihat Deskripsi Senjata
3. Pesan Feedback
4. Keluar

=====
>__<
=====

>> 4

Process finished with exit code 0
```

Gambar 8. Program File SOLID saat memilih pilihan 4

```
=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Username
2. Lihat Deskripsi Senjata
3. Tambah Buff Damage (Decorator)
4. Pesan Feedback
5. Keluar

=====
>_<
=====

>> 1
```

Gambar 9. Program File Design Pattern saat memilih pilihan 1

```
=====
ALBION ONLINE - STAFF INFO
=====

Masukkan username:

=====
>_<
=====

>> Naufal
```

Gambar 10. Program File Design Pattern memasukkan nama pada pilihan 1

```
=====
ALBION ONLINE - STAFF INFO
=====

Halo Naufal! (ENTER untuk kembali)

=====
>_<
=====
```

Gambar 11. Program File Design Pattern setelah memilih pilihan 1


```
=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Username
2. Lihat Deskripsi Senjata
3. Tambah Buff Damage (Decorator)
4. Pesan Feedback
5. Keluar

=====
>_<
=====

>> 2
```

Gambar 12. Program File Design Pattern saat memilih pilihan 2

```
=====
ALBION ONLINE - STAFF INFO
=====

Pilih Senjata:

1. Novice Fire Staff
2. Adept Holy Staff
3. Expert Great Staff

=====
>_<
=====

>> 1
```

Gambar 13. Program File Design Pattern memilih senjata di pilihan 2

```
=====
ALBION ONLINE - STAFF INFO
=====

Nama : Novice Fire Staff
Deskripsi : Staff api dasar untuk pemula
Base Damage : 10.0
Bonus Damage : 0.0
Total Damage : 10.0
Skill : Fire Bolt → Menembakkan api kecil ke musuh

(ENTER untuk kembali)

=====
>_<
=====
```

Gambar 14. Program File Design Pattern setelah memilih pilihan 2

```
=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Username
2. Lihat Deskripsi Senjata
3. Tambah Buff Damage (Decorator)
4. Pesan Feedback
5. Keluar

=====
>_<
=====

>> 3
```

Gambar 15. Program File Design Pattern saat memilih pilihan 3

```
=====
ALBION ONLINE - STAFF INFO
=====

Pilih Staff untuk diberi buff damage:

1. Novice Fire Staff
2. Adept Holy Staff
3. Expert Great Staff

=====
>_<
=====

>> 1
```

Gambar 16. Program File Design Pattern memilih senjata di pilihan 3

```
=====
ALBION ONLINE - STAFF INFO
=====

Masukkan jumlah bonus damage (angka, mis. 5):

=====
>_<
=====

>> 5
```

Gambar 17. Program File Design Pattern memasukkan angka di pilihan 3

```

=====
ALBION ONLINE - STAFF INFO
=====

Buff berhasil diterapkan!

Nama : Novice Fire Staff
Deskripsi : Staff api dasar untuk pemula
Base Damage : 10.0
Bonus Damage : 5.0
Total Damage : 15.0
Skill : Fire Bolt + Buff(+5) → Menembakkan api kecil ke musuh (Damage +5)

(ENTER untuk kembali)

=====
>__<
=====

```

Gambar 18. Program File Design Pattern setelah memilih pilihan 3

```

=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Username
2. Lihat Deskripsi Senjata
3. Tambah Buff Damage (Decorator)
4. Pesan Feedback
5. Keluar

=====
>__<
=====

>> 4

```

Gambar 19. Program File Design Pattern saat memilih pilihan 4

```

=====
ALBION ONLINE - STAFF INFO
=====

Tulis feedback:

=====
>__<
=====

>>
Terima Kasih

```

Gambar 20. Program File Design Pattern menulis teks di pilihan 4

```

=====
ALBION ONLINE - STAFF INFO
=====

Feedback diterima:

Terima Kasih

(ENTER untuk kembali

=====
>_<
=====

```

Gambar 21. Program File Design Pattern setelah memilih pilihan 4

```

=====
ALBION ONLINE - STAFF INFO
=====

1. Masukkan Username
2. Lihat Deskripsi Senjata
3. Tambah Buff Damage (Decorator)
4. Pesan Feedback
5. Keluar

=====
>_<
=====

>> 5

Process finished with exit code 0

```

Gambar 22. Program File Design Pattern saat memilih pilihan 5

C. Pembahasan

Penerapan Solid

Single Responsibility Principle

Prinsip Single Responsibility menyatakan bahwa kelas harusnya memiliki satu tanggung jawab utama. Pada kelas UI hanya memiliki satu tugas, dan mematuhi prinsip ini karena fungsinya hanya sebatas menangani interaksi antara sistem dan pengguna. Kelas ini bertugas menampilkan menu, menerima input, dan menyampaikan keluaran tanpa melakukan proses logika bisnis, pengelolaan senjata, atau manipulasi data.

Open/Closed & Liskov Substitution Principle

Kelas Staff.java menerapkan prinsip Open/Closed karena dirancang sebagai kelas abstrak yang dapat diperluas melalui subclass baru tanpa mengharuskan perubahan pada struktur kelas aslinya. Ketika ingin menambahkan staff baru maka hanya menambahkan kelas turunan saja tanpa memodifikasi kode Staff.java itu sendiri. Selain itu, Staff.java juga memenuhi Liskov Substitution Principle, sebab seluruh subclass seperti novicesFireStaff, adeptHolyStaff dan expertGreatStaff dapat digunakan sebagai objek Staff tanpa mengubah perilaku program dengan memperluas fungsi dari Staff melalui detail khusus seperti damage, deskripsi, dan skill tanpa mengubah definisi kelas induk.

Interface Segregation Principle

Interface Skill.java dirancang minimalis dengan hanya dua metode inti, yaitu untuk mendapatkan nama skill dan menggunakan skill. Karena hanya berisi fungsi yang benar-benar diperlukan oleh setiap skill, interface ini tidak membebani implementasi

Dependency Inversion Principle

Dependency Inversion Principle merupakan dimana modul tingkat tinggi tidak bergantung pada class konkret, pada Interface weaponRepository merupakan bentuk nyata dari prinsip Dependency Inversion, Semua layanan yang membutuhkan data senjata bergantung pada interface ini, bukan pada kelas repository yang spesifik. Kelas weaponService.java menerapkan prinsip

Dependency Inversion dengan menerima dependency melalui interface `weaponRepository`, bukan implementasinya secara langsung.

Penerapan Design Pattern

Factory Method Pattern

Dalam program ini, Factory Method diterapkan melalui class `staffFactory` yang bertanggung jawab menghasilkan objek `staff` dengan produk yang dihasilkan yaitu `noviceFireStaff`, `adpetHolyStaff`, dan `expertGreatStaff`. Dengan Factory Method, proses instansiasi objek tidak lagi dilakukan secara langsung di dalam menu, melainkan di kelas khusus yang menangani pembuatan objek. ketika jenis `staff` baru ditambahkan cukup dengan menambahkan subclass baru dan memperbarui `factory` tanpa mengubah logika di bagian lain.

Decorator Pattern

Decorator Pattern digunakan untuk menambahkan kemampuan atau modifikasi perilaku objek tanpa mengubah struktur kelas aslinya. Pada program ini, decorator diaplikasikan untuk memberikan buff atau peningkatan status pada `weapon`, misalnya meningkatkan damage melalui kelas seperti `buffDamageDecorator` sebagai pembungkus yang memberikan peningkatan damage, kepada objek `staff` asli. Decorator mengoverload metode tertentu, seperti `getBaseDamage()`, sehingga ia dapat menambah nilai buff sambil tetap mempertahankan perilaku objek awal.

Strategy Pattern

Strategy Pattern diterapkan untuk memisahkan perilaku `skill` dari objek `Staff`, sehingga setiap `skill` menjadi strategi yang independen. Interface *Skill* mendefinisikan perilaku dasar, sementara implementasinya, seperti *NoviceFireSkill*, *AdeptHolySkill*, dan *ExpertGreatSkill*, menjadi strategi yang dapat dipertukarkan. Dalam kelas `Staff`, atribut bertipe `Skill` disediakan sehingga setiap senjata dapat menggunakan `skill` yang berbeda tanpa mengubah kode internal `Staff`. perubahan atau penambahan `skill` baru tidak mempengaruhi kode `Staff` maupun `weapon` lain, cukup menambah kelas baru yang mengimplementasikan `Skill`.

TAUTAN GIT

<https://github.com/yuuukosoo/Praktikum-Pemrograman-II>